



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення

Лабораторна робота №2
«Діаграма варіантів використання.
Сценарії варіантів використання.
Діаграми UML. Діаграми класів.
Концептуальна модель системи»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третякова Ельвіра

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості	2
Тема	4
Схема прецеденту.....	4
Сценарії використання для 3 прецедентів	5
1. <i>Прецедент: Створення зображення.....</i>	5
2. <i>Прецедент: Редагування зображення</i>	6
3. <i>Прецедент: Збереження зображення.....</i>	7
Діаграма класів	8
Шаблон репозиторію	11
Код на GitHub	12
Структура бази даних	13
Висновок	14

Тема: Діаграма варіантів використання. Сценарії варіантів використання.

Діаграми UML. Діаграми класів. Концептуальна модель системи

Мета: Проаналізувати тему, намалювати схему прецеденту, діаграму класів, розробити основні класи і структуру бази.

Хід роботи

Короткі теоретичні відомості

1. Діаграма варіантів використання (Use Case Diagram)

Діаграма варіантів використання — це візуалізація функцій системи з позиції її користувачів (акторів). Вона показує:

- *Актори*: Особи або інші системи, які взаємодіють із вашою системою (наприклад, користувачі, адміністратори).

- *Варіанти використання*: Основні функції, які система надає акторам (наприклад, реєстрація, авторизація, перегляд інформації).

- *Зв'язки*: Лінії, що з'єднують акторів з варіантами використання, вказуючи на доступні їм функції.

2. Сценарії варіантів використання (Use Case Scenarios)

Сценарії варіантів використання описують конкретні взаємодії актора з системою для досягнення певної цілі. Кожен сценарій включає:

- *Назву*: Лаконічну назву сценарію.
- *Актор*: Того, хто ініціює сценарій.
- *Передумови*: Умови, що мають бути виконані до початку сценарію.
- *Основний потік*: Кроки, що описують нормальний перебіг дій.
- *Альтернативні потоки*: Інші можливі варіанти виконання сценарію, що виникають через помилки або альтернативні дії.

3. Діаграми UML (Unified Modeling Language)

UML — це стандартизована мова моделювання для візуалізації, специфікації, розробки та документування програмних систем. UML-діаграми поділяються на:

- *Структурні діаграми*: Відображають структуру системи (наприклад, діаграми класів, компонентів, об'єктів).
- *Поведенкові діаграми*: Показують динамічну взаємодію системи (наприклад, діаграми варіантів використання, послідовності, активностей).

4. Діаграми класів (Class Diagrams)

Діаграми класів є основними структурними діаграмами в UML, що відображають класи системи, їх атрибути, методи і зв'язки між класами. Основні елементи:

- *Клас*: Представлений прямокутником з ім'ям класу, атрибутами та методами.
- *Атрибути*: Описують дані, які зберігаються в класі (наприклад, ім'я, вік).
- *Методи*: Вказують на дії, які клас може виконувати (наприклад, створення, редагування).
- *Зв'язки*: Включають асоціації, агрегації, композиції та наслідування, що показують взаємодію між класами.

5. Концептуальна модель системи

Концептуальна модель — це абстрактне представлення системи, яке зосереджується на основних компонентах та їх зв'язках без технічних деталей. Вона включає:

- *Сутності*: Головні об'єкти системи (наприклад, користувачі, продукти).
- *Асоціації*: Зв'язки між сутностями, що показують їхню взаємодію.
- *Властивості*: Характеристики сутностей.

Така модель допомагає зрозуміти загальну структуру та функції системи перед її реалізацією.

Тема

Вибір варіанту:

Варіант – 4: Графічний редактор (proxy, prototype, decorator, bridge, flyweight,

SOA) Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Схема прецеденту

Схема прецеденту, що відповідає темі “Графічний редактор”, зображена на рисунку 1

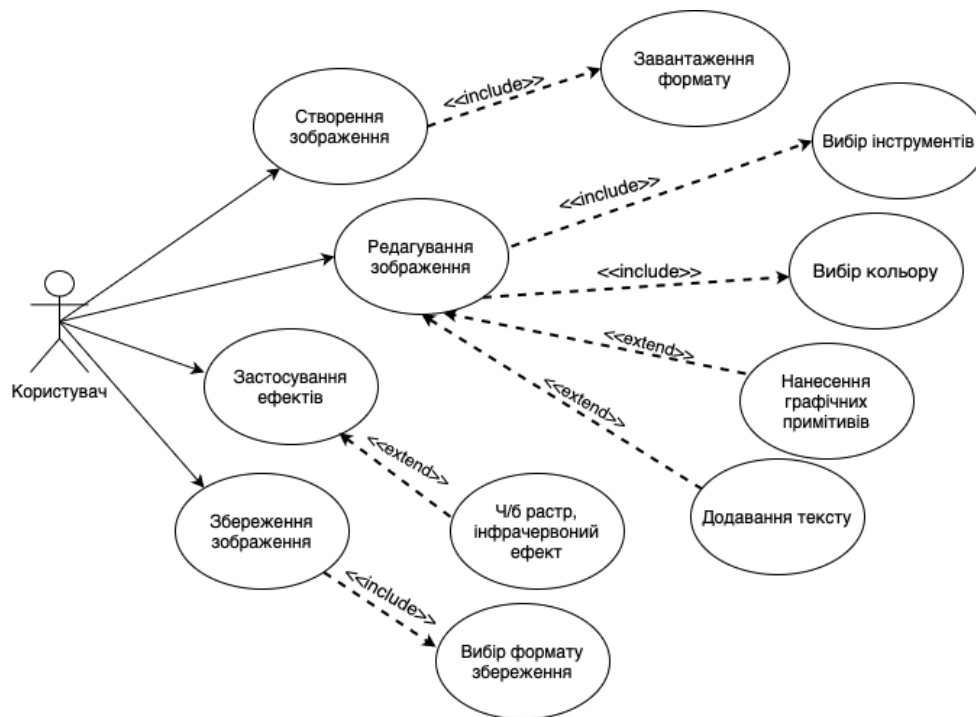


Рис.1

Ця схема прецеденту зображує основні етапи процесу обробки зображення.

1. Завантаження формату - перший крок, де користувач обирає формат для завантаження зображення.

2. Вибір інструментів - вибір необхідних інструментів для роботи із зображенням.

3. Вибір кольору - налаштування кольору, який буде використаний при роботі із зображенням.

4. Нанесення графічних примітивів - додавання елементів, таких як лінії, фігури тощо.

5. Додавання тексту - можливість додати текст до зображення.

6. Ч/Б растр, інфрачервоний ефект - застосування чорно-білого растрового або інфрачервоного ефекту для зображення.

7. Редагування зображення - процес редагування зображення, який включає зміну параметрів.

8. Застосування ефектів - додавання різноманітних ефектів для покращення зображення.

9. Збереження зображення - збереження зміненого зображення.

10. Вибір формату збереження - вибір формату, у якому буде збережено оброблене зображення.

Схема показує послідовність дій користувача при обробці зображення, включаючи вибір параметрів, редагування, застосування ефектів та збереження готового результату.

Сценарії використання для 3 прецедентів

Обираємо 3 прецеденти та описуємо сценарій використання:

1. Прецедент: Створення зображення

Актор: Користувач

Передумови:

- Користувач запустив графічний редактор.
- Користувач перебуває на головному екрані редактора.

Основний сценарій:

1. Користувач обирає опцію «Створити нове зображення».
2. Система запитує розміри нового зображення (ширина, висота).
3. Користувач вводить бажані розміри.
4. Система створює новий порожній холст з обраними розмірами.
5. Користувач переходить до редагування зображення.

Післяумови:

- Новий холст створений, і користувач може почати додавати графічні елементи.

Винятки:

- Якщо введені розміри некоректні (наприклад, нульові значення), система показує повідомлення про помилку та запитує коректні значення.

Альтернативний сценарій:

- Користувач обирає не нове зображення, а завантажує наявне зображення.
- Система відкриває вибране зображення у редакторі, дозволяючи подальше редагування.

2. Прецедент: Редагування зображення

Актор: Користувач

Передумови:

- Зображення вже завантажено або створене.
- Користувач знаходиться в режимі редагування.

Основний сценарій:

1. Користувач обирає інструмент (олівець, пензель, ластик).
2. Користувач обирає колір для інструменту.
3. Користувач додає графічні примітиви (лінії, кола) на зображення.
4. Користувач додає текст до зображення.
5. Користувач застосовує ефекти (чорно-білий растр, інфрачервоний ефект) до вибраних частин зображення.
6. Користувач задоволений редагуванням та переходить до збереження.

Післяумови:

- Всі зміни застосовані до зображення.

Винятки:

- Якщо інструмент не обраний, система повідомляє про помилку і пропонує обрати інструмент перед внесенням змін.
- Якщо користувач обирає некоректний формат для ефекту, система показує повідомлення про несумісність ефекту.

Альтернативний сценарій:

- Після вибору інструменту користувач вирішує повернутися до попереднього стану зображення, використовуючи функцію «Відміна» (Undo).
- Користувач продовжує редагування, змінюючи вибраний інструмент або колір.

3. Прецедент: Збереження зображення

Актор: Користувач

Передумови:

- Користувач завершив редагування зображення.
- Зображення готове до збереження.

Основний сценарій:

1. Користувач обирає опцію «Зберегти зображення».
2. Система пропонує вибрати формат для збереження (bmp, png, jpg).
3. Користувач обирає бажаний формат.
4. Система пропонує обрати місце для збереження файлу.
5. Користувач вибирає місце збереження та називає файл.
6. Система зберігає зображення у вибраному форматі.

Післяумови:

- Зображення збережене на пристрої користувача у вибраному форматі та місці.

Винятки:

- Якщо місце збереження недоступне або бракує місця, система повідомляє про це і пропонує вибрати інше місце.
- Якщо формат не підтримується, система просить вибрати інший формат для збереження.

Альтернативний сценарій:

- Користувач обирає опцію «Зберегти як», щоб зберегти копію зображення в іншому форматі або з іншою назвою.
- Система зберігає зображення у вибраному місці та форматі, не перезаписуючи оригінал.

Діаграма класів

Діаграма класів зображена на рисунку 2.

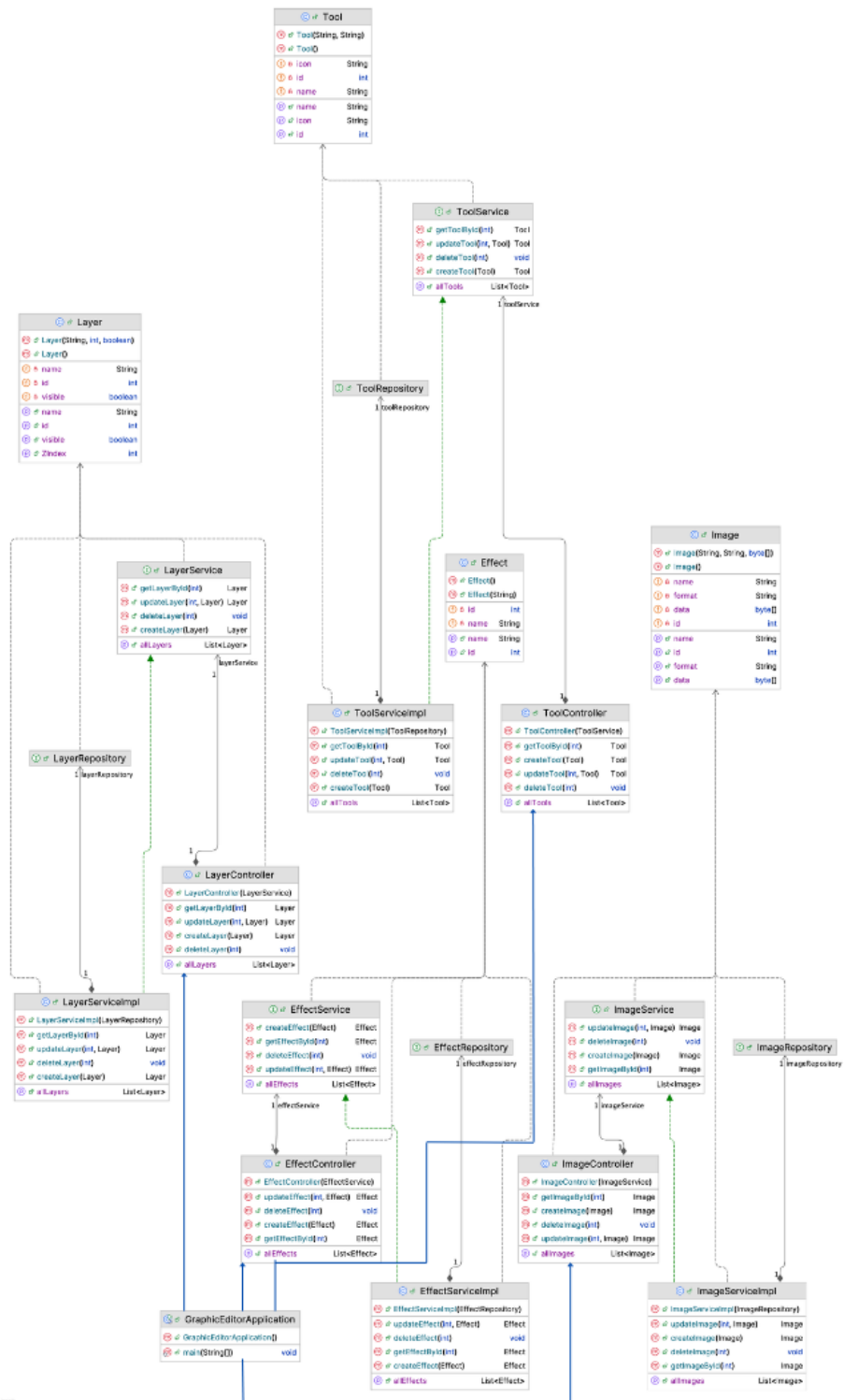


Рис.2

Ця діаграма класів представляє архітектуру для графічного редактора, який підтримує роботу зі зображеннями, шарами, інструментами та ефектами. Ось опис основних класів та їхніх взаємозв'язків:

Основні класи:

1. Image

- Зберігає дані про зображення (шлях до файлу, висота, ширина).
- Використовується для створення та збереження зображень у сховищі зображень.

2. ImageRepository

- Відповідає за збереження і завантаження об'єктів класу “Image”.
- Взаємодіє з “ImageService” для управління зображеннями.

3. ImageService

- Здійснює операції над зображеннями, зокрема отримання, збереження та видалення.
- Використовує “ImageRepository” для доступу до зображень.

4. Layer

- Представляє шар в графічному редакторі.
- Зберігає інформацію про розміри та позицію шару.

5. LayerService

- Виконує операції над шарами, такі як додавання, видалення і переміщення.
- Використовує “LayerRepository” для управління даними шарів.

6. LayerRepository

- Зберігає та керує об'єктами “Layer”.
- Використовується “LayerService” для зберігання і доступу до шарів.

7. Effect

- Містить базову інформацію про ефект (назва, тип).
- Використовується для застосування графічних ефектів до зображень.

8. EffectService

- Відповідає за виконання операцій над ефектами, включаючи додавання і видалення ефектів.
- Працює разом із “EffectsRepository” для управління ефектами.

9. EffectsRepository

- Керує об'єктами класу “Effect”.
- Взаємодіє з “EffectService” для збереження ефектів.

10. Tool

- Представляє інструмент графічного редактора (наприклад, пензель, ластик).
- Зберігає основну інформацію про інструмент, таку як назва та тип.

11. ToolService

- Відповідає за операції з інструментами, включаючи їх створення, видалення та збереження.
- Використовує “ToolRepository” для керування об'єктами “Tool”.

12. ToolRepository

- Зберігає та обробляє об'єкти “Tool”.
- Використовується “ToolService” для зберігання інструментів.

13. GraphicEditorAppSession

- Представляє сесію роботи в графічному редакторі.
- Зв'язує різні компоненти, такі як “ImageService”, “LayerService”, “ToolService” та “EffectService”.

Контролери:

1. ImageController

- Контролює операції над зображеннями в межах редактора.
- Використовує “ImageService” для маніпуляцій із зображеннями.

2. LayerController

- Контролює операції над шарами.
- Взаємодіє з “LayerService” для додавання, видалення і оновлення шарів.

3. ToolController

- Контролює операції над інструментами.
- Використовує “ToolService” для керування інструментами.

4. EffectController

- Контролює операції з ефектами.

- Використовує “EffectService” для додавання та застосування ефектів.

Зв’язки між класами:

- “GraphicEditorAppSession” зв’язує усі основні компоненти редактора, що дозволяє їм взаємодіяти між собою в рамках однієї сесії.
- Контролери (ImageController, LayerController, ToolController, EffectController) працюють як посередники між користувачем та сервісами, забезпечуючи доступ до функціоналу редактора.
- Сервіси (ImageService, LayerService, ToolService, EffectService) відповідають за основні операції над своїми об’єктами, тоді як репозиторії (ImageRepository, LayerRepository, ToolRepository, EffectsRepository) зберігають дані.

Ця структура дозволяє гнучко керувати основними елементами графічного редактора (зображеннями, шарами, інструментами, ефектами) і забезпечує розширюваність та масштабованість.

Шаблон репозиторію

Шаблон репозиторію зображено на рисунку 3.

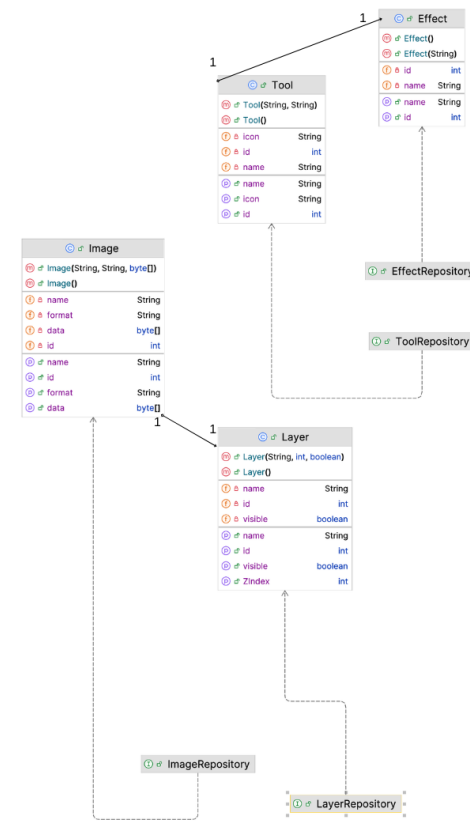


Рис.3

На зображенні показана структура класів і репозиторіїв для зберігання та управління даними об'єктів, які представляють зображення, шари, інструменти та ефекти в системі обробки зображень.

1. *ImageRepository* – відповідає за зберігання та управління об'єктами типу Image. Клас Image містить атрибути:

- id (int) – ідентифікатор зображення.
- name (String) – назва зображення.
- type (String) – тип або формат зображення.
- data (byte[]) – масив байтів, що містить дані зображення.

2. *LayerRepository* – відповідає за зберігання та управління об'єктами типу Layer. Клас Layer містить атрибути:

- id (int) – ідентифікатор шару.
- name (String) – назва шару.
- visible (boolean) – видимість шару.
- zIndex (int) – порядок шару за глибиною (для визначення черговості відображення).

3. *ToolRepository* – відповідає за зберігання та управління об'єктами типу Tool. Клас Tool містить атрибути:

- id (int) – ідентифікатор інструмента.
- name (String) – назва інструмента.
- description (String) – опис інструмента.

4. *EffectRepository* – відповідає за зберігання та управління об'єктами типу Effect. Клас Effect містить атрибути:

- id (int) – ідентифікатор ефекту.
- name (String) – назва ефекту.
- description (String) – опис ефекту.

Кожен з репозиторіїв надає інтерфейс для доступу до відповідних даних і є частиною загальної структури для роботи з зображеннями, де Image складається з кількох Layer, а Tool та Effect використовуються для застосування до зображень або їхніх шарів.

Код на GitHub

Код завантажений на : <https://github.com/Ellviraaaaaa/GraphicEditor.git>

Структура бази даних

Структура бази даних зображена на рисунку 4.

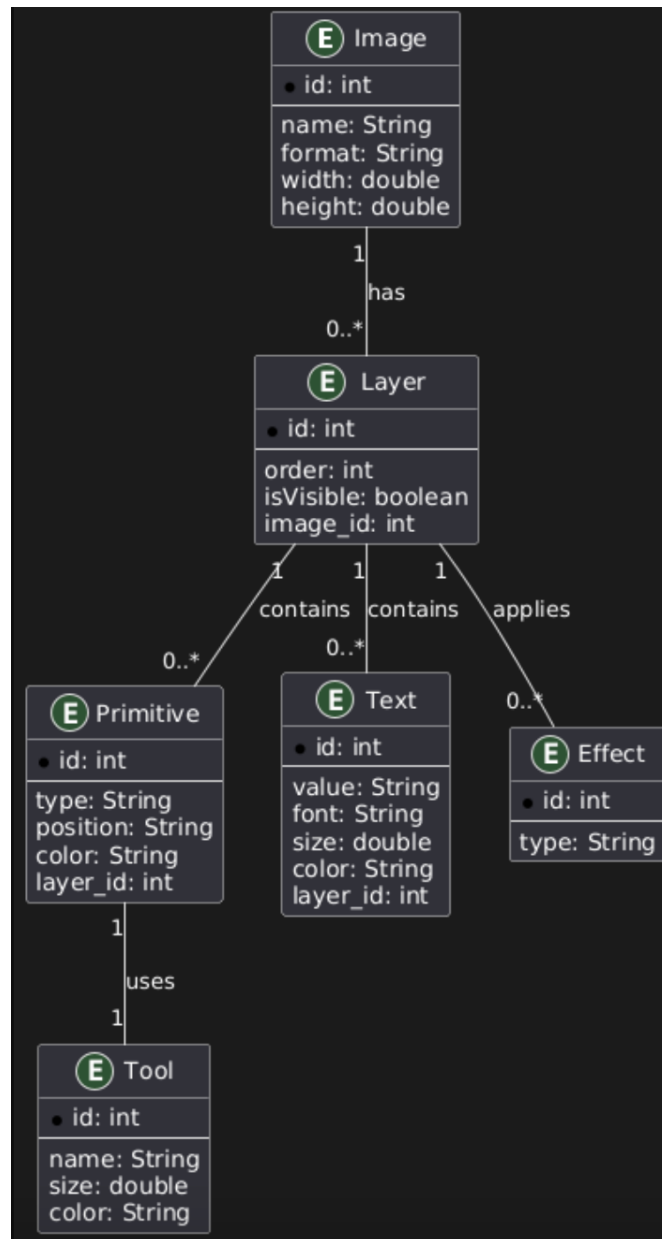


Рис.4

Структура бази даних організована для управління зображеннями, шарами, інструментами, примітивами, текстовими елементами та ефектами у графічному редакторі.

1. Image:

- Це основна таблиця, що зберігає базову інформацію про кожне зображення

- Таблиця Image має зв'язок «один до багатьох» із таблицею Layer, що дозволяє зберігати кілька шарів для одного зображення.

2. *Layer*:

- Таблиця Layer представляє окремі шари, що складають зображення.

3. *Primitive*:

- Таблиця Primitive зберігає дані про графічні примітиви (такі як коло, лінія або квадрат), що розташовуються на шарі.

4. *Text*:

- Таблиця Text містить текстові елементи, які можна додавати на шари зображення.

5. *Effect*:

- Таблиця Effect використовується для зберігання ефектів, що застосовуються до шарів.

6. *Tool*:

- Таблиця Tool зберігає інформацію про інструменти, що використовуються для створення або редагування елементів у зображенні.

Ця структура бази даних дозволяє ефективно організувати зображення, шари, інструменти, примітиви, текстові елементи та ефекти. Завдяки зв'язкам між таблицями забезпечується цілісність даних і можливість швидкої обробки змін, що є важливим для повнофункціонального графічного редактора.

Висновок

В даній лабораторній роботі я навчилась створювати схему прецедентів, робити опис прецедентів, а також ознайомилась та створила діаграму класів, ознайомилась та виконала структуру бази даних.